1. Fine-tuning Strategy

In this section, our goal is to identify a suitable base model to serve as *ClassifyLM* and fine-tune it for the task of KA classification using our curated dataset. We detail the baseline methods analyzed (Section 1.1), the evaluation metrics used for comparison (Section 1.2), and the results of our experimental analysis of the candidate models (Section 1.3).

1.1. Baselines

We consider the following NLP methods for the ClassifyLM task:

- Zero-shot Learning: In zero-shot learning, we simply *prompt* a LLM without domain-specific fine-tuning. We selected DeepSeek-V3 (Liu et al., 2024) and ChatGPT-4o-mini (OpenAI, 2023) for this baseline, since both models are widely available and relatively cost-effective. Additionally, we evaluate Qwen-2.5b-7B-Instruct (Team, 2024) (the model adopted for PreprocessLM), to motivate that different models were used for PreprocessLM and ClassifyLM. The specific prompt used for zero-shot classification is detailed in Figure 1 in 2.

- Random Forest (RF): RFs are a well-known traditional machine learning method. In contrast to deep learning methods that learn directly from raw text, RFs require feature extraction. Here, we use both Bag-of-Words (BoW) and Term-Frequency-Inverse Document Frequency (TF-IDF), that assign numerical values to words based on how frequently they appear in the training set.

- LSTM: To establish a direct comparative baseline with the most relevant related work by Dzurenda et al. (Dzurenda et al., 2024), we utilized the core architecture described in their paper. This model employs the standard BERT (Devlin et al., 2019) tokenizer for text processing, followed by a unidirectional Long Short-Term Memory (LSTM) network for the final classification. Note that it was not possible to use Dzurenda et al.'s trained model directly, as it was optimized for classifying 12 European Union Agency for Cybersecurity (ENISA) work profiles rather than the 9 CSEC2017 Knowledge Areas. Therefore, we adapted their architecture and trained it on our dataset with the recommended learning rate of 0.001 for 20 epochs.

- Fine-tuned Transformer Models: We experimented with several transformer variants, namely BERT (Devlin et al., 2019), ALBERT (Lan et al., 2019), RoBERTa (Liu et al., 2019), and DistilBERT (Sanh et al., 2019). BERT serves as a foundational benchmark. DistilBERT offers a more parameter-efficient alternative, reducing the model size by approximately 40% through knowledge distillation. RoBERTa improves upon BERT with optimized training techniques like dynamic masking and extended training, while ALBERT introduces mechanisms to minimize the memory footprint. All models were fine-tuned for a maximum of 20 epochs using a sigmoid output activation and the Binary Cross Entropy loss to accommodate the multi-label classification setting.

1.2. Metrics

The task of assigning course descriptions to one or more KAs is a multi-label, multi-class classification problem. While the accuracy provides an intuitive measure of overall prediction correctness, it is not well-suited for multi-label classification. It is highly affected by class imbalance: models can achieve a high accuracy solely by performing well on the KA with the highest number of samples. Accuracy also treats all errors equally and does not distinguish between false positives and false negatives, even though these could have different implications for curriculum analysis. Therefore, to enable a more nuanced performance evaluation, we do not use the accuracy, but instead compute the macro-averaged precision, recall, and F1 score. Here, 'macro-averaged' indicates that metrics are computed per KA and then averaged, such that KAs with fewer samples contribute as much to the final score as KAs with more samples. The definitions of the metrics used are as follows:

- Precision: The precision or *true positive rate* measures the proportion of correctly predicted positive labels among all predicted positives for a given class. We report the *macro-averaged precision*, which computes the metric independently for each KA and then takes the mean, assigning equal weight to all KAs. This is defined as:

$$Precision = \frac{1}{C} \sum_{i=1}^{C} \frac{TP_i}{TP_i + FP_i} \tag{1}$$

  Here, TP_i represents the number of True Positives (TPs) for KA i, while FP_i stands for the number of False Positives (FPs). C is the total number of classes, which is equal to 9 in the case of KAs. Macro-averaged precision yields a value between 0 and 1, where a high value indicates a low rate of false positives across the model's predictions.

- Recall: The recall or *sensitivity* quantifies the proportion of actual positives that have been correctly identified as positive by the model. Analogous to precision, we report the *macro-averaged recall*:

$$Recall = \frac{1}{C}\sum_{i=1}^{C}\frac{TP_i}{TP_i + FN_i} \tag{2}$$

Here, FN_i stands for the number of False Negatives (FNs) for a given KA i. The macro-averaged recall yields a value between 0 and 1, where a high recall can be achieved when the model has few false negatives.

- F1 score: The F1 score is the harmonic mean of the precision and the recall, addressing scenarios where one of these two metrics dominates. We report the *macro-averaged F1 score*, calculated as the mean of the per-class F1 scores:

$$\text{F1} = \frac{1}{C}\sum_{i=1}^{C}\frac{2\,TP_i}{2\,TP_i + FP_i + FN_i}. \tag{3}$$

The F1 score ranges from 0 to 1, where a higher value indicates superior performance by jointly considering precision and recall. Although the F1 score is the harmonic mean of precision and recall, its macro-average is not. Consequently, the macro-averaged F1 score can be lower than the macro-averaged precision and recall.

1.3. Model Selection

Table 1: The classification performance of various NLP methods on the fine-tuning dataset. Column 1 gives an overview of the NLP model class used, Column 2 provides model specifications. The performance metrics used are the macro-averaged Precision (0–1), Recall (0–1), and F1 score (0–1), as defined in Section 1.2 The reported scores were computed using 10-folds cross validation and have been averaged over 5 random seeds.

Category	Specification	Precision (↑)	Recall (↑)	F1 score (↑)
Zero-shot	Qwen-2.5b-7B-Instruct (Team, 2024)	0.29	0.44	0.32
	ChatGPT-4o-mini (OpenAI, 2023)	0.40	0.33	0.29
	DeepSeek-V3 (Liu et al., 2024)	0.38	0.49	0.41
Random Forest	Bag-of-words	0.67	0.44	0.53
	TF-IDF	0.71	0.44	0.54
LSTM	BERT tokenizer (Dzurenda et al., 2024)	0.14	0.13	0.13
Transformers	ALBERT (Lan et al., 2019)	0.68	0.48	0.55
	DistilBERT (Sanh et al., 2019)	0.71	0.57	0.62
	RoBERTa (Liu et al., 2019)	0.69	0.58	0.63
(selected)⟶	**BERT (Devlin et al., 2019)**	**0.71**	**0.59**	**0.64**

Table 1 gives an overview of the cross-validation scores of all baseline NLP methods mentioned in Section 1.1, as computed on the fine-tuning dataset (described in our main paper). To ensure a robust performance estimate, we use *k-fold cross validation*. This technique partitions the fine-tuning dataset, consisting of synthetic CSEC2017 data and NIST Knowledge Descriptions as described in the previous subsection, into k non-overlapping folds. In each of the k iterations, the model is trained on $k-1$ folds (corresponding to 90% of the data for $k = 10$) and validated on the hold-out fold (the remaining 10%). In this way, it is possible to compute the performance of the model on the entire fine-tuning dataset. To balance the bias and variance trade-off associated with this process, the recommended setting for k is 10. We adopt $k = 10$ and split the training dataset into 10 non-overlapping 90/10 train/val splits.

We first evaluate zero-shot classification performance using Qwen-2.5b-7B-Instruct (which also serves as the backbone for PreprocessLM), and the state-of-the-art DeepSeek-V3 and ChatGPT-4o-mini models. An identical prompt, as indicated in Figure 1 in 2, was used for all models to ensure a fair comparison. As mentioned in earlier sections, we hypothesized that the tasks designed for PreprocessLM and ClassifyLM cannot be executed by the same model. From the data in Table 1 we can see that the Qwen-2.5b-7B-Instruct model does not have a good off-the-shelf performance for KA classification, a limitation that cannot be remedied via fine-tuning due to the small size of our dataset. The more powerful zero-shot DeepSeek-V3 and ChatGPT-4o-mini models also achieve low scores, demonstrating the limitations of general-purpose LLMs without domain-specific fine-tuning.

Random Forest (RF) based methods showed moderate improvements over zero-shot LLM prompting, which suggests that the features extracted from the fine-tuning dataset possess predictive value. Nonetheless, the limited performance of the RF models indicates that these models struggle to capture the complex semantics inherent

to cybersecurity concepts. The baseline method proposed by Dzurenda et al. (2024), based on a unidirectional LSTM trained on pre-tokenized text, performed significantly worse than all other approaches. However, this can in part be attributed to the fact that the hyperparameter settings and training methodology were not optimized for our dataset, but set to the default values recommended by Dzurenda et al. (2024). A bidirectional LSTM or a deeper neural network architecture could have improved the performance of the LSTM-based approach, but these architectural changes are beyond the proposed methodology of Dzurenda et al. and therefore considered outside of the scope of this paper.

In contrast, Transformer-based architectures substantially outperformed other approaches, with BERT achieving an F1 score of 0.64. We attribute this superior performance to the self-attention mechanism in transformer models, which allows them to weigh the importance of all words in a context simultaneously, capturing long-range contextual information more effectively than LSTMs or RFs. While transformer-based models generally outperformed traditional methods, BERT emerged as the best candidate. Based on these results, we therefore designate the fine-tuned BERT model as ClassifyLM.

2. Zero-shot classification prompt

We employed zero-shot prompting of ChatGPT-mini-4o, and DeepSeek-V3 as a baseline for comparison with the proposed ClassifyLM structure (see Section 1.1). Zero-shot prompting requires the selected model to perform Knowledge Area annotation based solely on the instructions and definitions provided in the prompt, without any prior examples. The prompt provided to both ChatGPT-mini-4o and DeepSeek-V3 is detailed in Figure 1.

Prompt Structure for Knowledge Area Classification

System Prompt:

```
"You are a helpful AI assistant.
Instructions:
a. Carefully read the knowledge statement.
b. Choose one or more of the following (0, 1, 2, 3, 4, 5, 6, 7, 8).
c. Do NOT include any explanation or additional text in the response."
```

User Message:

```
"#Question: Classify the following statement {knowledge} into one or multiple of
the following knowledge areas:
Options: {"0": "miscellaneous (this includes Computer Science, Business
and Law, Communication and Networking, Information Technology, Cyberspace
Practice, Pedagogy, and Intelligence)", "1": "data security", "2": "software
security", "3": "component security", "4": "connection security", "5":
"system security", "6": "human security", "7": "organizational security",
"8": "societal security"}"
```

Figure 1: The prompt template used for annotating knowledge statements with Knowledge Areas using Zero-Shot prompting of DeepSeek-V3 or ChatGPT-4o-mini. The placeholder {knowledge} is replaced by a NICE Knowledge Description or a synthetically derived CSEC2017 topic.

References

Devlin, J., Chang, M.W., Lee, K., Toutanova, K., 2019. Bert: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers), pp. 4171–4186.

Dzurenda, P., Ricci, S., Sikora, M., Stejskal, M., Lendák, I., Adão, P., 2024. Enhancing cybersecurity curriculum development: Ai-driven mapping and optimization techniques, in: Proceedings of the 19th International Conference on Availability, Reliability and Security, pp. 1–10.

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., Soricut, R., 2019. ALBERT: A lite BERT for self-supervised learning of language representations. ArXiv:1909.11942. Available at: `https://arxiv.org/abs/1909.11942` (Accessed: 2025-11-28).

Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., 2024. Deepseek-v3 technical report. ArXiv:2412.19437. Available at: `https://arxiv.org/pdf/2412.19437` (Accessed: 2025-11-28).

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., 2019. RoBERTa: A robustly optimized BERT pretraining approach. ArXiv:1907.11692. Available at: `https://arxiv.org/pdf/1907.11692` (Accessed: 2025-11-28).

OpenAI, 2023. Chatgpt-4. `https://chatgpt.com/`. Accessed: 2025-04-24.

Sanh, V., Debut, L., Chaumond, J., Wolf, T., 2019. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. ArXiv:1910.01108. Available at: https://arxiv.org/pdf/1910.01108 (Accessed: 2025-11-28).

Team, Q., 2024. Qwen2.5: A party of foundation models. URL: https://qwenlm.github.io/blog/qwen2.5/. accessed: 2025-11-28.